

Suitably Impressive Thesis Title



Candidate Number: 1234567

University of Oxford

Word Count: 15000

A thesis submitted for the degree of

Master of Science in Advanced Computer Science

Trinity 2026

Abstract

Your abstract goes here. Check your departmental regulations, but this is usually expected to be under 300 words — roughly one page.

A workable structure is one paragraph per move: (1) context and why the problem matters, (2) the gap, (3) what you did, (4) what you found, and (5) what it means more broadly.

Acknowledgements

Thank your supervisors first, then anyone whose ideas or feedback shaped the direction of the work.

Then colleagues, collaborators, and friends.

Then any funding or institutional support you received.

Finally, family.

Contents

1	Introduction	1
1.1	Problem Setting and Research Gap	2
1.2	Proposed Approach and Main Findings	3
1.3	Contributions	4
1.4	Thesis Overview	5
2	Background	6
2.1	Generative Modelling	6
2.1.1	Conditional Generative Modelling	7
2.1.2	Inverse Problems	7
2.1.3	Relation to Reward Alignment	8
2.2	Flow-Based Generative Modelling	9
2.2.1	Normalising Flows	9
2.2.2	Flow Models and Flow Matching	15
2.2.3	Flow Maps	18
2.3	Variational Inference	20
2.3.1	VI for a Single Observation	20
	Bibliography	22
A	Supplementary Material	26
A.1	Change of Variables	26
A.2	Score functions and score matching	26
A.3	Additional Experiments	26
A.3.1	GMM Experiments	26

List of Figures

Chapter 1

Introduction

One goal of generative modelling is to learn how to produce novel samples that resemble those drawn from an unknown data distribution p_{data} . Diffusion and flow models [1–5] have been highly successful at this task, achieving state-of-the-art results in image, video and audio generation [6–8]. Both use the same high-level framework for generation: samples $\mathbf{z} \sim p_{\text{base}}$ drawn from a simple noise distribution are transformed continuously over time towards the data distribution [9].

Flow and diffusion models specify the dynamics of this continuous transformation between noise and data as governed by an ordinary differential equation (ODE) and a stochastic differential equation (SDE), respectively [4, 10]. An ODE defines a path from a noise to data sample by specifying the local velocity at each point along it. In a flow model, a neural network is trained to predict this velocity. Generating data therefore consists of iteratively advancing a sample from noise to data through a sequence of small steps, querying the neural network at each step to determine the direction of travel. Consequently, although flow models enable high-fidelity generation, they can often require hundreds of neural network evaluations [3], which can be computationally prohibitive.

Flow maps [11, 12] address this limitation by training a neural network to predict the cumulative displacement between any two times along the noise-to-data paths defined by the ODE, rather than the local velocity at a single point. A flow map can therefore transport a noise sample directly to a data sample in a single neural network evaluation. Flow maps can be learned by bootstrapping from a pre-trained flow model via distillation [11, 13]. However, it is also possible to train a flow map from scratch, as demonstrated by MeanFlow [14].

1.1 Problem Setting and Research Gap

Many applications require conditional generation: producing samples from the data distribution that satisfy an external condition, such as a class label or text prompt [6, 15, 16]. A canonical example of conditional generation is inverse problem solving [17], in which an observation $\mathbf{y}$ is generated by corrupting an unknown data sample $\mathbf{x} \sim p_{\text{data}}$ according to $\mathbf{y} = \mathcal{A}(\mathbf{x}) + \boldsymbol{\eta}$, where $\mathcal{A}$ is a known forward operator and $\boldsymbol{\eta}$ represents measurement noise. In image inverse problems, for example, $\mathcal{A}$ may downsample, blur or mask an image. Since these operations typically discard information, multiple data samples may be consistent with the same observation. The goal of inverse problem solving is therefore to draw samples from the posterior $p_{\text{data}}(\mathbf{x} \mid \mathbf{y})$, which represents the distribution of plausible data samples consistent with the given observation $\mathbf{y}$.

One approach to conditional generation is to train a generative model specifically for the conditional task, either from scratch or by fine-tuning a pretrained unconditional model [16, 18]. This can provide fast conditional generation, including one-step generation with a flow map [14, 19]. However, adapting the generative model to each new conditional generation setting typically requires a new round of training or fine-tuning, which can be prohibitively expensive for large generative models.

Guidance-based methods offer an alternative for flow (and diffusion) models. These methods keep the pretrained unconditional flow model frozen but nudge its velocity prediction at each sampling step, thereby steering the path from noise towards a data sample that satisfies the given condition [20–23]. This approach hence allows for same unconditional model to be reused across different conditional generation settings without fine-tuning.

However, this guidance strategy does not transfer naturally to one-step generation with a flow map. A flow map transports an initial noise sample directly to a data sample in a single evaluation, leaving no intermediate predictions through which guidance can be applied. The key insight of Variational Flow Maps (VFM) [24] is that generation can instead be controlled through the choice of starting noise. For a one-step unconditional flow map, the starting noise is the only remaining lever for controlling its output. Conditional generation can therefore be reframed as learning a distribution over starting noise samples that the flow map sends to data samples satisfying the given condition.

Let $p_{\theta}(\mathbf{x})$ denote the model distribution (which approximates $p_{\text{data}}(\mathbf{x})$) obtained by drawing $\mathbf{z} \sim p_{\text{base}}$ and passing it through the flow map, where θ denotes

the neural-network parameters of the flow map. For a given condition $\mathbf{y}$, VFM formulates the required noise distribution as the noise-space posterior $p_\theta(\mathbf{z} \mid \mathbf{y})$. Intuitively, this distribution is obtained by pulling the corresponding model posterior $p_\theta(\mathbf{x} \mid \mathbf{y})$ backwards through the flow map into noise space.

Although $p_\theta(\mathbf{z} \mid \mathbf{y})$ cannot be sampled from directly, its density can be evaluated up to a normalising constant, making variational inference (VI) [25] a natural approach for approximating it. VI introduces a tractable distribution $q_\phi(\mathbf{z} \mid \mathbf{y})$, called a variational distribution, with parameters ϕ , to approximate this posterior. Fitting $q_\phi(\mathbf{z} \mid \mathbf{y})$ using the variational objective requires both being able to sample from it and evaluate its density. VFM satisfies these requirements by parameterising $q_\phi(\mathbf{z} \mid \mathbf{y})$ as a diagonal Gaussian whose mean and variance are predicted from $\mathbf{y}$ by a neural network, termed the noise adapter.

Although a Gaussian supports both sampling and exact density evaluation, it may be too simple to approximate the potentially complex noise-space posterior $p_\theta(\mathbf{z} \mid \mathbf{y})$. VFM compensates for this by jointly training the noise adapter and the flow map, allowing the geometry of the noise space to be reshaped such that $p_\theta(\mathbf{z} \mid \mathbf{y})$ can be more easily approximated by a simple Gaussian distribution. VFM therefore enables one-step conditional generation, but only by fine-tuning the flow map. This leaves open whether one-step conditional generation can be achieved while keeping the unconditional flow map fixed.

1.2 Proposed Approach and Main Findings

The central question of this project is whether a more expressive variational distribution $q_\phi(\mathbf{z} \mid \mathbf{y})$ can approximate $p_\theta(\mathbf{z} \mid \mathbf{y})$ while leaving the pretrained flow map frozen. We investigate this question by replacing VFM’s Gaussian noise adapter with a conditional normalising flow [26, 27]. Normalising flows support both sampling and exact density evaluation, satisfying the requirements of variational training while being capable of representing substantially more complex distributions than a Gaussian. When amortising over observations $\mathbf{y}$, the inverse-problem setting also naturally permits the approximating distribution, $q_\phi(\mathbf{z} \mid \mathbf{y})$, to be learned via maximum likelihood estimation. We therefore introduce a training objective that combines the variational loss with a maximum-likelihood loss [28], and show that the combination is more effective than either loss alone.

The resulting approach enables one-step conditional generation using an

existing pretrained unconditional flow map while keeping its parameters fixed. Solving each new conditional generation setting using this approach requires training only a lightweight noise adapter. We evaluate the method across inverse problems of increasing difficulty, progressing from a two-dimensional checkerboard problem to image brightness, $4\times$ super-resolution and inpainting. On the two-dimensional problem, the Gaussian adapter fails to approximate the noise-space posterior accurately, whereas the normalising-flow adapter captures its multimodal structure. For $4\times$ super-resolution, our method achieves the lowest maximum mean discrepancy (MMD) among all evaluated baselines, including VFM.

However, we find that the approach is not competitive on image inpainting. Our analysis suggests that the main difficulty is learning, in a single step, which initial noise samples will map to images matching an observation that constrains many pixel values. This motivates training the adapter to predict a point partway along the noise-to-data path, rather than the initial noise itself, before mapping that point to data in a single flow-map evaluation. We leave the development of this extension to future work.

1.3 Contributions

This thesis makes the following contributions:

1. We adapt the Variational Flow Maps (VFM) framework for one-step conditional generation so that it is training-free with respect to the flow map. In particular we replace the Gaussian noise adapter with a more expressive normalising flow. To our knowledge, this is the first work to investigate solving conditional generation entirely within the noise space of a pretrained unconditional flow map without fine-tuning the flow map itself.
2. We further extend VFM by introducing a training objective that combines both the variational loss with a maximum-likelihood loss. We show that this combined objective is more effective than training the noise adapter with either maximum likelihood or the variational loss alone.
3. We introduce a principled evaluation framework that isolates error attributable to the trained noise adapter from error attributable to the frozen pretrained flow map.

4. We demonstrate that the more expressive normalising flow adapter can substitute for flow map fine-tuning on two-dimensional checkerboard, image-brightness and $4\times$ super-resolution inverse problems. On super-resolution, our method achieves the lowest MMD among all evaluated baselines, including VFM. However, we find that it is not competitive for image inpainting, and we provide a detailed analysis of its failure.

1.4 Thesis Overview

Chapter 2

Background

This chapter introduces the background required to understand the proposed method. We first formalise the goals of generative modelling and conditional generative modelling. We then introduce inverse problems as the conditional generation setting considered in this thesis and relate them to the broader framework of reward alignment. Next, we review the flow-based generative models on which the project builds upon: normalising flows, flow models trained through flow matching, and flow maps. Finally, we introduce variational inference and Variational Flow Maps (VFM), which our approach directly builds on.

2.1 Generative Modelling

Suppose we wish to generate examples of a particular class of object. This class may be defined as broadly or narrowly as we like. We restrict our attention to objects that can be represented as vectors $\mathbf{x} \in \mathbb{R}^D$. In generative modelling, we assume that there exists a target probability distribution $p_{\text{target}}(\mathbf{x})$ which places high density on vectors corresponding to plausible examples of the specified class of object. The goal of generative modelling is to learn an algorithm that can produce samples from this target distribution.

Often, the target distribution is a data distribution $p_{\text{data}}(\mathbf{x})$ over, for example, images, videos or molecular structures. In this setting, we typically have access to a finite dataset $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^N$ of samples from the data distribution. Using this dataset, the aim is to learn a model distribution $p_{\theta}(\mathbf{x})$ that approximates $p_{\text{data}}(\mathbf{x})$. Depending on the generative modelling algorithm, it may support sampling from $p_{\theta}(\mathbf{x})$, evaluating its density, or both.

Many generative models, such as generative adversarial networks (GANs)

[29] and the flow-based generative models [4, 14, 27] considered in this thesis, generate data by first drawing noise $\mathbf{z} \sim p_{\text{base}}$ from a simple base distribution and then transforming it into a data sample $\mathbf{x}$. The learned transformation should induce a model distribution $p_{\theta}(\mathbf{x})$ that approximates the target distribution.

2.1.1 Conditional Generative Modelling

Often, we do not wish to generate an arbitrary sample from the target distribution. Instead, we wish to generate samples that satisfy a given condition $\mathbf{y} \in \mathbb{R}^M$, where $\mathbf{y} \sim p_Y(\mathbf{y})$. For example, p_Y may be a distribution over class labels, or text prompts. This task is known as conditional generative modelling. For a given condition $\mathbf{y}$, the task can be formalised as sampling from the conditional target distribution $p_{\text{target}}(\mathbf{x} \mid \mathbf{y})$ [10].

Typically, we do not want an algorithm that can generate samples for only one fixed condition. Instead, the aim is to learn an algorithm that can generate samples for different conditions $\mathbf{y} \sim p_Y(\mathbf{y})$. For example, an image generator may be required to accept different text prompts.

2.1.2 Inverse Problems

Inverse problem solving is a canonical example of conditional generation and is the focus of this thesis. To define an inverse problem, we first define the corresponding forward problem. Let $\mathbf{x} \sim p_{\text{data}}(\mathbf{x})$ be a data sample and let $\mathcal{A} : \mathbb{R}^D \rightarrow \mathbb{R}^M$ be a known forward operator. The forward operator acts on $\mathbf{x}$ to produce an observation

$$\mathbf{y} = \mathcal{A}(\mathbf{x}) + \boldsymbol{\eta}, \quad \boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, \sigma_y^2 \mathbf{I}_M) \quad (2.1)$$

where $\boldsymbol{\eta}$ represents measurement noise. In image inverse problems, for example, $\mathcal{A}$ may blur, downsample or mask an image. The forward problem consists of computing the observation $\mathbf{y}$ from the data sample $\mathbf{x}$ [17].

The inverse problem is the reverse problem. Given an observation $\mathbf{y}$, the aim is to infer the unknown data sample $\mathbf{x}$ from which it was produced. However, the forward operator typically destroys information and is therefore not invertible. Several data samples may consequently be consistent with the same observation. Therefore, bayesian inverse problem solving instead aims to draw samples from

the posterior distribution

$$p_{\text{data}}(\mathbf{x} \mid \mathbf{y}) = \frac{p(\mathbf{y} \mid \mathbf{x})p_{\text{data}}(\mathbf{x})}{p_Y(\mathbf{y})} \quad (2.2)$$

where $p(\mathbf{y} \mid \mathbf{x})$ is the likelihood of observing $\mathbf{y}$ given $\mathbf{x}$. Under the Gaussian measurement model in Equation 2.1, the likelihood is

$$p(\mathbf{y} \mid \mathbf{x}) = \mathcal{N}(\mathbf{y} \mid \mathcal{A}(\mathbf{x}), \sigma_y^2 \mathbf{I}_M) \propto \exp\left(-\frac{\|\mathbf{y} - \mathcal{A}(\mathbf{x})\|_2^2}{2\sigma_y^2}\right) \quad (2.3)$$

Substituting the likelihood into Equation 2.2 gives

$$p_{\text{data}}(\mathbf{x} \mid \mathbf{y}) \propto p_{\text{data}}(\mathbf{x}) \exp\left(-\frac{\|\mathbf{y} - \mathcal{A}(\mathbf{x})\|_2^2}{2\sigma_y^2}\right) \quad (2.4)$$

The posterior therefore assigns high density to samples that are both plausible under $p_{\text{data}}(\mathbf{x})$ and consistent with the observation $\mathbf{y}$.

2.1.3 Relation to Reward Alignment

Conditional generation can be viewed as a particular instance of the broader task of reward alignment. This connection allows us to discuss general approaches for reward alignment in the related-work chapter with the view that they can be applied to conditional generation tasks, such as inverse problem solving.

Reward alignment aims to generate samples from the target distribution with specified desirable properties, as quantified by a reward function $r(\mathbf{x})$. For example, a reward function may compute an aesthetic rating of an image. Reward alignment can be formalised as sampling from

$$p_{\text{reward}}(\mathbf{x}) \propto p_{\text{target}}(\mathbf{x}) \exp(r(\mathbf{x})) \quad (2.5)$$

This distribution reweights the target distribution towards samples with higher reward, and is known as the reward-tilted distribution [30–32].

To express conditional generation as reward alignment, we let each condition $\mathbf{y}$ define a corresponding reward function $r_{\mathbf{y}}(\mathbf{x})$. In particular, we choose this reward to be the log-likelihood of the condition given the sample $r_{\mathbf{y}}(\mathbf{x}) = \log p(\mathbf{y} \mid \mathbf{x})$. The corresponding reward-tilted distribution is

$$p_{\text{reward}}(\mathbf{x} \mid \mathbf{y}) \propto p_{\text{target}}(\mathbf{x})p(\mathbf{y} \mid \mathbf{x}) \propto p_{\text{target}}(\mathbf{x} \mid \mathbf{y}) \quad (2.6)$$

For the inverse problems defined above, $p(\mathbf{y} \mid \mathbf{x})$ is the likelihood in Equation 2.3. In this example, samples receive a high reward when they are consistent with the observation. Inverse problem solving is therefore a particular instance of

conditional generation, which can itself be formulated as reward alignment [24].

2.2 Flow-Based Generative Modelling

Flow-based generative models transform samples from a tractable base distribution $p_{\text{base}}(\mathbf{z})$ into samples from a target distribution $p_{\text{target}}(\mathbf{x})$. Normalising flows represent this transformation as a finite composition of invertible maps [27]. Flow models and flow maps represent it as a continuous transformation governed by an ODE. While flow models learn the vector field defining the ODE [4, 10], flow maps instead aim to directly learn the integral of the ODE [33].

These models play different roles in this thesis. A pretrained flow map acts as the generative model that maps noise to data. Whereas we use a normalising flow to learn a conditional distribution over the noise space of the flow model (i.e. the noise adapter). Flow models provide the continuous-time transport framework from which flow maps are defined.

2.2.1 Normalising Flows

We first define normalising flows and explain how their densities are evaluated. We then introduce autoregressive flows, which are the specific variation of normalising flows that we use to implement the noise adapter. Finally we explain how normalising flows can be learned using forward or reverse KL divergence objectives. Our presentation and terminology follow the review of Papamakarios et al. [27], to which we refer for proofs and a broader survey of architectures.

Definition and Change of Variables

Suppose we wish to model a target distribution $p_{\text{target}}(\mathbf{x})$. A normalising flow begins with a simple base distribution $p_{\text{base}}(\mathbf{z})$ from which we can easily sample and whose density can be evaluated exactly. It then learns a transformation $T_\phi : \mathbb{R}^D \rightarrow \mathbb{R}^D$ that maps samples from this base distribution to samples whose distribution approximates the target distribution. More precisely, samples are generated according to

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z}) \quad \mathbf{x} = T_\phi(\mathbf{z}) \quad \mathbf{x} \sim p_\phi(\mathbf{x}) \quad (2.7)$$

and the parameters ϕ of the transformation are learned such that $p_\phi(\mathbf{x})$ approximates $p_{\text{target}}(\mathbf{x})$ [27].

A normalising flow requires T_ϕ to be a diffeomorphism: an invertible, continuously differentiable map whose inverse is also continuously differentiable. Invertibility means that every point $\mathbf{x}$ corresponds to a unique base point $\mathbf{z} = T_\phi^{-1}(\mathbf{x})$.

Because T_ϕ is a diffeomorphism, the density of p_ϕ is well-defined and can be evaluated using the change-of-variables formula

$$p_\phi(\mathbf{x}) = p_{\text{base}}\left(T_\phi^{-1}(\mathbf{x})\right) \left| \det \frac{\partial T_\phi^{-1}(\mathbf{x})}{\partial \mathbf{x}} \right| \quad (2.8)$$

To evaluate the density according to Equation 2.8, the inverse transformation must not only exist but also be computationally tractable. Intuitively, Equation 2.8 states that in order to compute the density of $\mathbf{x} \sim p_\phi$, first consider mapping $\mathbf{x}$ back to its corresponding base point $\mathbf{z} = T_\phi^{-1}(\mathbf{x})$. Subsequently, the density at $\mathbf{x}$ is equal to the base density at $\mathbf{z}$, weighted according to the local change in volume produced by T_ϕ . If T_ϕ expands the space around $\mathbf{z}$, the same probability mass is spread across a larger region and the density decreases. If it contracts the space, the density increases.

When $\mathbf{x}$ has been generated from a known base point $\mathbf{z}$, the same density can instead be evaluated using the Jacobian of the forward transformation

$$p_\phi(\mathbf{x}) = p_{\text{base}}(\mathbf{z}) \left| \det \frac{\partial T_\phi(\mathbf{z})}{\partial \mathbf{z}} \right|^{-1} \quad \mathbf{x} = T_\phi(\mathbf{z}) \quad (2.9)$$

This form avoids evaluating T_ϕ^{-1} because the corresponding base point $\mathbf{z}$ is already known. Both forms are needed in this thesis: the inverse form in Equation 2.8 underlies the forward-KL objective, and this forward form underlies the reverse-KL objective, both of which we use to train the noise adapter and introduce later.

A normalising flow therefore supports both sampling and exact density evaluation. However, for an unrestricted $D \times D$ Jacobian, computing the determinant in Equation 2.8 requires $\mathcal{O}(D^3)$ operations.

Constructing Expressive and Tractable Flows

In practice, T_ϕ is parameterised by a neural network. Since an arbitrary neural network is neither guaranteed to be invertible nor to have a tractable Jacobian determinant, its functional form must be restricted to guarantee these properties, without limiting its ability to represent complex target distributions.

A common approach is to construct T_ϕ as a composition of L relatively simple

invertible transformations, which we refer to as flow layers

$$T_\phi = T_{\phi_L} \circ T_{\phi_{L-1}} \circ \dots \circ T_{\phi_1} \quad (2.10)$$

where $\phi = (\phi_1, \dots, \phi_L)$. Starting from $\mathbf{z}_0 = \mathbf{z}$, each layer is applied in turn as $\mathbf{z}_\ell = T_{\phi_\ell}(\mathbf{z}_{\ell-1})$, with $\mathbf{z}_L = \mathbf{x}$. Importantly, a composition of diffeomorphisms is itself a diffeomorphism, whose inverse is obtained by applying the inverse layers in reverse order.

By the chain rule, the Jacobian determinant of this composition is the product of the Jacobian determinants of its layers, so its log-determinant is the sum of the layers' log-determinants. Hence, if each layer has an efficient inverse and Jacobian determinant, so does the overall flow. Stacking more layers increases the expressiveness of T_ϕ , while the cost of evaluating its log-determinant grows only linearly with L [27].

Several approaches have been developed for constructing flow layers that are invertible, expressive and with cheap Jacobian determinant. These include autoregressive flows, linear flows and residual flows [27]. We focus on autoregressive flows because they have been shown to scale effectively to generative modelling for high-dimensional data, including images [34].

Autoregressive Flow Layers

Consider a single flow layer T_{ϕ_ℓ} . To simplify notation, denote its input by $\mathbf{z} \in \mathbb{R}^D$ and its output by $\mathbf{z}' = T_{\phi_\ell}(\mathbf{z}) \in \mathbb{R}^D$. An autoregressive flow layer splits the coordinates of $\mathbf{z}$ into K equal-sized blocks $\mathbf{z}_1, \dots, \mathbf{z}_K$. The output $\mathbf{z}'$ is split into corresponding blocks $\mathbf{z}'_1, \dots, \mathbf{z}'_K$. Each output block $\mathbf{z}'_i$ is obtained by applying an elementwise transformation to the corresponding input block $\mathbf{z}_i$. The parameters of this transformation may depend only on the preceding input blocks $\mathbf{z}_{<i} = (\mathbf{z}_1, \dots, \mathbf{z}_{i-1})$. In particular,

$$\mathbf{z}'_i = \tau(\mathbf{z}_i; \mathbf{h}_i) \quad \mathbf{h}_i = c_i(\mathbf{z}_{<i}) \quad i = 1, \dots, K \quad (2.11)$$

The function τ , called the transformer, performs the elementwise transformation of $\mathbf{z}_i$. Its parameters $\mathbf{h}_i$ are predicted by the i th conditioner c_i , which is typically implemented using a neural network. Here, transformer refers to the function that transforms each block, rather than the Transformer neural-network architecture.

For the layer to be invertible, the transformer τ must be invertible with respect to $\mathbf{z}_i$ when its parameters $\mathbf{h}_i$ are fixed. This is achieved by making τ strictly monotonic in $\mathbf{z}_i$. The conditioners, by contrast, need not be invertible.

Under these conditions, the layer can be inverted block by block: $\mathbf{h}_1$ depends on no input blocks, so $\mathbf{z}_1 = \tau^{-1}(\mathbf{z}'_1; \mathbf{h}_1)$, and each subsequent $\mathbf{z}_i$ can be recovered once $\mathbf{z}_{<i}$ is known [27]. Inverting T_{ϕ_ℓ} therefore requires K sequential steps, each evaluating the conditioner and the transformer.

Because τ acts elementwise and $\mathbf{h}_i$ depends only on $\mathbf{z}_{<i}$, each output block depends only on its own input block and the preceding ones, so the Jacobian of the layer is lower-triangular. Its log-determinant is therefore a sum of D diagonal terms, computable in $\mathcal{O}(D)$ time [27]. This matters for the noise adapter, whose noise space has dimension $D = 3072$ for CIFAR-10, where an $\mathcal{O}(D^3)$ determinant at every training step would be prohibitive.

Computational Direction: IAF and MAF

Sampling from a normalising flow applies each layer T_{ϕ_ℓ} in turn, whereas evaluating a density via Equation 2.8 applies the inverse layers. We showed above that inverting an autoregressive layer requires K sequential steps. By contrast, the forward transformation $\mathbf{z}' = T_{\phi_\ell}(\mathbf{z})$ can be evaluated in parallel. Given $\mathbf{z}$, all input blocks are already known, so every $\mathbf{h}_i = c_i(\mathbf{z}_{<i})$ can be computed simultaneously. In practice, the conditioners share one neural network that maps $\mathbf{z}$ to all of $\mathbf{h}_1, \dots, \mathbf{h}_K$ in a single forward pass, with its weights masked to remove any path from $\mathbf{z}_{\geq i}$ to $\mathbf{h}_i$ [27, 35]. Every output block $\mathbf{z}'_i = \tau(\mathbf{z}_i; \mathbf{h}_i)$ can then also be computed simultaneously. Sampling is therefore comparatively inexpensive, whereas density evaluation is more expensive.

If an application instead prioritises fast density evaluation, we can instead allow the parameters to be computed from the output blocks, $\mathbf{h}_i = c_i(\mathbf{z}'_{<i})$. With this change, the layer remains invertible with a lower-triangular Jacobian, but the costs are reversed: $T_{\phi_\ell}^{-1}$ can now be evaluated in parallel, whereas T_{ϕ_ℓ} requires K sequential steps.

Computing the parameters from the input blocks $\mathbf{z}_{<i}$ gives an inverse autoregressive flow (IAF) [36], which provides fast sampling. Computing them from the output blocks $\mathbf{z}'_{<i}$ gives a masked autoregressive flow (MAF) [37], which provides fast density evaluation. This distinction matters for the noise adapter, since reverse-KL training relies on sampling, whereas forward-KL training relies on density evaluation.

Transformer Autoregressive Flows

We now introduce the normalising-flow architecture used to represent the noise adapter in this project: Transformer Autoregressive Flow (TarFlow) [34]. TarFlow constructs the overall normalising flow as a composition of L masked autoregressive flow layers. It adapts the original MAF architecture [37] to better model distributions over images. It achieves this through four main design choices. We explain each of these choices in turn.

The first design choice determines how the image-shaped variable $\mathbf{z} \in \mathbb{R}^{C \times H \times W}$, with C channels, height H and width W , is divided into blocks. TarFlow splits the image into non-overlapping $P \times P$ patches spanning all channels, and flattens each patch into a block of CP^2 coordinates, which it calls a token. This gives $K = (H/P)(W/P)$ tokens, initially ordered by scanning the image from left to right and top to bottom.

The second design choice specifies the transformer τ applied to each token. TarFlow uses the elementwise affine transformation

$$\tau(\mathbf{z}_i; \mathbf{h}_i) = \exp(\boldsymbol{\alpha}_i) \odot \mathbf{z}_i + \boldsymbol{\beta}_i \quad \mathbf{h}_i = (\boldsymbol{\alpha}_i, \boldsymbol{\beta}_i) \quad (2.12)$$

where $\boldsymbol{\alpha}_i, \boldsymbol{\beta}_i \in \mathbb{R}^{CP^2}$ and $\odot$ denotes elementwise multiplication. The exponential ensures that every scale is positive, so the transformation is invertible.

The third design choice specifies the conditioner. Recall that, in a masked autoregressive flow, the parameters of token i are computed as $\mathbf{h}_i = c_i(\mathbf{z}'_{<i})$. Rather than using a separate network for each c_i , TarFlow computes all of $\mathbf{h}_1, \dots, \mathbf{h}_K$ with a single causal Transformer network, with one such network per flow layer. This computation proceeds in three steps. First, each token $\mathbf{z}'_i \in \mathbb{R}^{CP^2}$ is projected to the hidden dimension d of the Transformer by a learned matrix $W_{\text{in}} \in \mathbb{R}^{d \times CP^2}$, and a learned positional embedding $\mathbf{p}_i \in \mathbb{R}^d$ is added, giving the embedding $\mathbf{e}_i$. Second, the sequence $\mathbf{e}_1, \dots, \mathbf{e}_K$ is processed by **Transformer**, which denotes a stack of Transformer blocks, each consisting of causal multi-head self-attention followed by a multilayer perceptron. This produces an output $\mathbf{u}_i \in \mathbb{R}^d$ at every position. Third, a learned matrix W_{out} maps the output at position $i - 1$ to the affine parameters of token i . In summary,

$$\begin{aligned} \mathbf{e}_i &= W_{\text{in}} \mathbf{z}'_i + \mathbf{p}_i \\ (\mathbf{u}_1, \dots, \mathbf{u}_K) &= \text{Transformer}(\mathbf{e}_1, \dots, \mathbf{e}_K) \\ \mathbf{h}_i &= (\boldsymbol{\alpha}_i, \boldsymbol{\beta}_i) = W_{\text{out}} \mathbf{u}_{i-1} \end{aligned} \quad (2.13)$$

Because of the causal attention mask, the output $\mathbf{u}_i$ depends only on the tokens

$\mathbf{z}'_{\leq i}$. Computing $\mathbf{h}_i$ from the output at the preceding position, $\mathbf{u}_{i-1}$, therefore ensures that $\mathbf{h}_i$ depends only on $\mathbf{z}'_{< i}$, as a masked autoregressive flow requires. The first token has no preceding position, so its parameters are set to $\mathbf{h}_1 = (\mathbf{0}, \mathbf{0})$. By Equation 2.12, this gives $\mathbf{z}'_1 = \exp(\mathbf{0}) \odot \mathbf{z}_1 + \mathbf{0} = \mathbf{z}_1$, so each flow layer passes its first token through unchanged.

The fourth design choice concerns the token ordering. TarFlow reverses the order of the tokens between successive flow layers, so that tokens which appear early in one layer appear late in the next. Since each token is conditioned only on the tokens before it, this reversal allows every token to be conditioned on every other token across layers. Together, these design choices result in a scalable normalising flow architecture for modelling image distributions. In our image inverse problem experiments, we choose TarFlow to implement our noise adapter because the noise adapter models a distribution over the image-shaped noise space of a flow map.

Training Normalising Flows

So far, we have described how to construct a normalising flow T_ϕ that induces a model distribution $p_\phi(\mathbf{x})$. We now discuss how to learn ϕ such that $p_\phi(\mathbf{x})$ approximates $p_{\text{target}}(\mathbf{x})$. A natural approach is to minimise the KL divergence between the two distributions. Since the KL divergence is asymmetric, it can be minimised in either the forward or the reverse direction.

We first define the forward KL loss as

$$\begin{aligned} \mathcal{L}_{\text{fKL}}(\phi) &:= D_{\text{KL}}(p_{\text{target}} \parallel p_\phi) = \mathbb{E}_{\mathbf{x} \sim p_{\text{target}}} \left[\log \frac{p_{\text{target}}(\mathbf{x})}{p_\phi(\mathbf{x})} \right] \\ &= -\mathbb{E}_{\mathbf{x} \sim p_{\text{target}}} [\log p_\phi(\mathbf{x})] + \text{const} \end{aligned} \quad (2.14)$$

The constant is the expected log target density, which does not depend on ϕ . Minimising $\mathcal{L}_{\text{fKL}}$ is therefore equivalent to maximising the expected log-likelihood assigned by the normalising flow to samples from the target distribution. This is precisely maximum-likelihood training. Estimating this loss requires samples $\mathbf{x} \sim p_{\text{target}}$ and the ability to evaluate $p_\phi(\mathbf{x})$ at those samples. By the change-of-variables formula in Equation 2.8, evaluating $p_\phi(\mathbf{x})$ requires computing $T_\phi^{-1}(\mathbf{x})$ and its Jacobian determinant. Importantly, it does not require evaluating the target density p_{target} .

We next define the reverse KL loss as

$$\mathcal{L}_{\text{rKL}}(\phi) := D_{\text{KL}}(p_\phi \parallel p_{\text{target}}) = \mathbb{E}_{\mathbf{x} \sim p_\phi} [\log p_\phi(\mathbf{x}) - \log p_{\text{target}}(\mathbf{x})] \quad (2.15)$$

The expectation in Equation 2.15 is taken over $\mathbf{x} \sim p_\phi$. Since p_ϕ is obtained by transforming the base distribution through T_ϕ , the standard reparameterisation trick [38] allows this expectation to be rewritten over $\mathbf{z} \sim p_{\text{base}}$. Substituting the forward change-of-variables formula 2.9 then gives

$$\begin{aligned}\mathcal{L}_{\text{rKL}}(\phi) &= \mathbb{E}_{\mathbf{z} \sim p_{\text{base}}} [\log p_\phi(T_\phi(\mathbf{z})) - \log p_{\text{target}}(T_\phi(\mathbf{z}))] \\ &= \mathbb{E}_{\mathbf{z} \sim p_{\text{base}}} \left[\log p_{\text{base}}(\mathbf{z}) - \log \left| \det \frac{\partial T_\phi(\mathbf{z})}{\partial \mathbf{z}} \right| - \log p_{\text{target}}(T_\phi(\mathbf{z})) \right] \quad (2.16)\end{aligned}$$

Equation 2.16 shows that reverse-KL training requires sampling from p_{base} , evaluating T_ϕ and its Jacobian determinant, and evaluating the target density at the generated sample. Unlike forward-KL training, it does not require samples from p_{target} . Furthermore, an unnormalised target density is sufficient because its unknown normalising constant is independent of ϕ and therefore does not affect optimisation [27].

The two objectives also behave differently when p_{target} is multimodal. Forward KL takes its expectation over p_{target} , so any region where p_{target} has mass but p_ϕ assigns little density incurs a very large penalty $-\log p_\phi(\mathbf{x})$. Forward KL is therefore mode-covering: it forces p_ϕ to cover every mode, even at the cost of placing mass between modes. Reverse KL takes its expectation over p_ϕ , so it penalises placing mass where p_{target} has none, but not missing a mode. Reverse KL is therefore mode-seeking, and often concentrates p_ϕ on a subset of the target’s modes [28].

2.2.2 Flow Models and Flow Matching

Flow models describe the transformation from noise to data as a continuous-time process, rather than as a finite chain of invertible maps. Instead of learning the entire transformation at once, a flow model only needs to specify how a sample should move locally at each point in time. We index time by $t \in [0, 1]$ and adopt the convention of MeanFlow [14]: a sample is drawn from the base distribution at $t = 1$ and is continuously transformed into a sample from the target distribution at $t = 0$. The definitions and results in this subsection follow Holderrieth and Erives [10] and Lipman et al. [4].

ODEs and Trajectories

The local motion of a point is specified by a vector field $v : \mathbb{R}^D \times [0, 1] \rightarrow \mathbb{R}^D$, which assigns a velocity $v(\mathbf{x}, t)$ to every location $\mathbf{x}$ and time t . To describe how

a point starting at $\mathbf{z}$ moves under this vector field, let $\mathbf{x}_t$ denote its location at time t . Its motion is governed by the ordinary differential equation (ODE)

$$\frac{d\mathbf{x}_t}{dt} = v(\mathbf{x}_t, t) \quad \mathbf{x}_1 = \mathbf{z} \quad (2.17)$$

which states that the point starts at $\mathbf{z}$ at time $t = 1$ and, as t decreases, always moves with the velocity that the vector field specifies at its current location and time. The resulting path $(\mathbf{x}_t)_{t \in [0,1]}$ is called a trajectory, and is the solution of the ODE.

If v is continuously differentiable with bounded derivatives, the Picard–Lindelöf theorem guarantees that, for every starting point $\mathbf{z}$, exactly one trajectory satisfies Equation 2.17 [10]. Each starting point therefore determines a single unique trajectory, and hence a single end point $\mathbf{x}_0$. This uniqueness is what later makes flow maps well defined.

Flow Models

The ODE is deterministic, so randomness enters only through the starting point. We denote random variables by capital letters. Suppose the starting point is drawn at random, $\mathbf{X}_1 \sim p_{\text{base}}$, and then follows the vector field according to Equation 2.17. For each t , its location $\mathbf{X}_t$ along the resulting trajectory is a deterministic function of the random starting point, and is therefore itself a random variable, with some distribution p_t . As t decreases from 1 to 0, following the vector field therefore transports $p_1 = p_{\text{base}}$ through a continuous sequence of distributions p_t , ending at p_0 .

A flow model parameterises the vector field by a neural network v_θ with parameters θ . Since our ultimate goal is to sample from p_{target} , θ is learned so that the distribution of the end point $\mathbf{X}_0$, which we denote p_θ , approximates p_{target} . Following v_θ from fresh noise $\mathbf{X}_1 \sim p_{\text{base}}$ then produces new samples distributed according to an approximation of p_{target} at time $t = 0$.

Sampling

In practice, v_θ cannot be followed exactly, because the ODE has no closed-form solution when the vector field is a neural network. Instead, the trajectory is followed in discrete steps. The simplest scheme is Euler’s method, which starts from $\mathbf{x}_1 = \mathbf{z}$ with $\mathbf{z} \sim p_{\text{base}}$, divides $[0, 1]$ into n steps of size $h = 1/n$, and at

each step moves the sample a short distance along its current velocity:

$$\mathbf{x}_{t-h} = \mathbf{x}_t - h v_\theta(\mathbf{x}_t, t) \quad t = 1, 1-h, \dots, h \quad (2.18)$$

The velocity is subtracted because time runs from $t = 1$ to $t = 0$. Smaller steps follow v_θ more accurately, but each step costs one network function evaluation (NFE), so accurate sampling can require many evaluations. This is the cost that flow maps are designed to remove.

Flow Matching

We want to learn a vector field v_θ that transports $\mathbf{X}_1 \sim p_{\text{base}}$ to $\mathbf{X}_0 \sim p_\theta$, such that p_θ approximates p_{target} . However, this requirement only constrains the two ends of the transformation. Many different vector fields transport p_{base} to p_{target} , and they differ in the intermediate distributions p_t that the samples pass through on the way. Flow matching [4] removes this ambiguity by explicitly constructing a target sequence of distributions p_t , called a probability path, together with a vector field that moves samples along it. That is, if we draw samples from p_{base} and follow this constructed vector field, the samples are distributed according to the constructed probability path p_t at every time t . This constructed vector field is called the marginal vector field, and flow matching trains v_θ to match it.

In the flow matching framework, constructing p_t boils down to choosing a ‘conditional probability path’. We choose the CondOT conditional probability path in this thesis, and refer to Holderrieth and Erives [10] for details regarding the exact construction of p_t . Given this choice, p_t is very simple. A sample from p_t is obtained by drawing a data point $\mathbf{x}_0 \sim p_{\text{target}}$ and a noise sample $\mathbf{x}_1 \sim p_{\text{base}}$ independently, and taking the point a fraction t of the way along the straight line from $\mathbf{x}_0$ to $\mathbf{x}_1$:

$$\mathbf{x}_t = (1-t)\mathbf{x}_0 + t\mathbf{x}_1 \quad (2.19)$$

At $t = 0$ this point is the data sample and at $t = 1$ it is the noise sample, so $p_0 = p_{\text{target}}$ and $p_1 = p_{\text{base}}$, as required. Importantly, this straight line is only a way of drawing samples from p_t . It does not describe the trajectories of the marginal vector field, which are generally curved rather than straight.

Having fixed p_t , flow matching trains v_θ using a simple regression objective. We sample a time $t \sim \mathcal{U}[0, 1]$, a data point $\mathbf{x}_0 \sim p_{\text{target}}$ and a noise sample $\mathbf{x}_1 \sim p_{\text{base}}$, construct $\mathbf{x}_t$ using Equation 2.19, and regress the output of the

network at $\mathbf{x}_t$ against the velocity of the straight line, $\mathbf{x}_1 - \mathbf{x}_0$:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, \mathbf{x}_0, \mathbf{x}_1} \|v_\theta(\mathbf{x}_t, t) - (\mathbf{x}_1 - \mathbf{x}_0)\|^2 \quad (2.20)$$

In practice, this expectation is estimated by averaging over a minibatch of sampled triples $(t, \mathbf{x}_0, \mathbf{x}_1)$. Although the training algorithm is simple, showing that its minimiser is the marginal vector field is fairly involved. We refer readers to Holderrieth and Erives [10] and Lipman et al. [4] for the proof.

2.2.3 Flow Maps

Our presentation of flow maps follows Dieleman [33] and Boffi, Albergo, and Vanden-Eijnden [11].

Motivation and Definition

Recall from Section 2.2.2 that generating a sample with a flow model requires simulating a trajectory from noise to data in many small Euler steps, each of which requires one evaluation of the velocity network v_θ . Sampling from a flow model is therefore expensive.

Consider a point on a trajectory, at location $\mathbf{x}_s$ at time s . If we follow the vector field v exactly from time s to another time t , taking infinitesimally small steps along the trajectory, the point $\mathbf{x}_s$ is displaced by exactly the integral $\int_s^t v(\mathbf{x}_\tau, \tau) d\tau$. By taking small steps in the direction output by the velocity network, as in Euler’s method, we are effectively approximating this integral. This motivates training a neural network to learn the integral of the marginal vector field directly, rather than the vector field itself. In particular, we want to learn the flow map

$$F(\mathbf{x}_s, s, t) = \mathbf{x}_s + \int_s^t v(\mathbf{x}_\tau, \tau) d\tau \quad (2.21)$$

which takes as input the location $\mathbf{x}_s$ of a point at time s on a trajectory and outputs the location $\mathbf{x}_t$ at time t on that same trajectory. If a neural network F_θ has been successfully trained to predict the flow map, we can draw $\mathbf{z} \sim p_{\text{base}}$ and evaluate the one-step generator $G_\theta(\mathbf{z}) := F_\theta(\mathbf{z}, 1, 0)$ to obtain a sample at $t = 0$. Sampling therefore requires only a single forward pass of the network.

Well-Definedness and the Noise–Data Bijection

For the flow map to be a well-defined function, the inputs $\mathbf{x}_s$, s and t must determine exactly one output. Two things must therefore hold. First, the point

$\mathbf{x}_s$ at time s must lie on only one trajectory, since otherwise it would be unclear which trajectory to follow. Second, following that trajectory to time t must lead to only one location. Both follow from the Picard–Lindelöf theorem introduced in Section 2.2.2.

A consequence of the Picard–Lindelöf theorem is that the vector field v defines a bijection between noise samples $\mathbf{z} \sim p_{\text{base}}$ and target samples $\mathbf{x} \sim p_{\text{target}}$. Intuitively, following the vector field from a noise sample $\mathbf{z}$ according to Equation 2.17 leads deterministically to exactly one target sample $\mathbf{x}$, and following it backwards in time from $\mathbf{x}$ leads back to exactly one noise sample, namely $\mathbf{z}$. VFM relies on this bijection: it pulls a conditional distribution over data back into noise space via the deterministic vector field. Because the pairing between noise and target samples is one to one, this procedure gives a well-defined distribution over noise. This would not be the case for diffusion models, whose sampling procedure is stochastic.

Training Flow Maps with MeanFlow

Training a flow map is harder than training a flow model. A flow model describes the paths between noise and data only locally, through the velocity at each point, whereas a flow map must describe them globally, through the displacement between any two points on the same path [33].

The predominant methods for training flow maps are derived from a simple rule that any valid flow map must satisfy, such as the Eulerian or Lagrangian consistency rule [11, 33]. These rules relate the flow map to the underlying marginal vector field v , so the training objectives derived from them require access to v . Methods differ in how v is provided. Distillation methods first train a flow model and use it to provide v [12, 13], whereas more recent methods avoid the need for a pretrained flow model and train the flow map from scratch.

MeanFlow [14] is one such method. Its training objective, which we denote $\mathcal{L}_{\text{MF}}$, is derived from the Eulerian consistency rule, and we give the derivation in Appendix ???. The derivation relies on an alternative parameterisation of the flow map, through the average velocity over the interval from s to t :

$$V(\mathbf{x}_s, s, t) = \frac{1}{t - s} \int_s^t v(\mathbf{x}_\tau, \tau) \, \mathrm{d}\tau \quad (2.22)$$

$$F(\mathbf{x}_s, s, t) = \mathbf{x}_s + (t - s) V(\mathbf{x}_s, s, t) \quad (2.23)$$

Since each parameterisation can be obtained from the other, the two are equivalent. Since MeanFlow has shown to be state-of-the-art, we use flow maps trained with

MeanFlow in all of our experiments: for the two-dimensional experiments, and a publicly released MeanFlow model for the image experiments.

2.3 Variational Inference

In Section ??, we will see that VFM formulates conditional generation with a flow map as inferring a posterior distribution over the noise space of the flow map. This posterior distribution can only be evaluated up to a normalising constant. Variational inference (VI) is the standard tool for approximating such distributions [25]. This section introduces VI, first for a single observation and then in its amortised form, which both VFM and our method use. Our presentation roughly follows Murphy [38].

2.3.1 VI for a Single Observation

Consider a probabilistic model with an unobserved latent variable $\mathbf{z}$, distributed according to a prior $p(\mathbf{z})$. In this model, an observation $\mathbf{y}$ is generated from a latent $\mathbf{z} \sim p(\mathbf{z})$ by sampling $\mathbf{y} \sim p(\mathbf{y} | \mathbf{z})$, where $p(\mathbf{y} | \mathbf{z})$ is called the likelihood. Suppose we are given an observation $\mathbf{y}$ and wish to determine which latent generated it. More precisely, in probabilistic modelling, we wish to determine the following posterior over latents which could have plausibly generated $\mathbf{y}$

$$p(\mathbf{z} | \mathbf{y}) = \frac{p(\mathbf{y} | \mathbf{z}) p(\mathbf{z})}{p(\mathbf{y})} \quad p(\mathbf{y}) = \int p(\mathbf{y} | \mathbf{z}) p(\mathbf{z}) d\mathbf{z} \quad (2.24)$$

Computing $p(\mathbf{y})$ requires integrating over all latents, which is generally intractable. We therefore cannot evaluate $p(\mathbf{z} | \mathbf{y})$ exactly, nor sample from it. To overcome this, VI chooses a tractable family of distributions $q_\psi(\mathbf{z})$, parameterised by variational parameters ψ , and optimises ψ so that $q_\psi(\mathbf{z})$ approximates $p(\mathbf{z} | \mathbf{y})$. Importantly, the variational family must allow both density evaluation and sampling. A common choice is the family of Gaussian distributions, in which case ψ consists of a mean and a variance.

VI optimises ψ by minimising the reverse KL divergence between $q_\psi(\mathbf{z})$ and $p(\mathbf{z} | \mathbf{y})$ with respect to ψ . Using the definition of KL divergence (twice) and

Bayes' rule,

$$\begin{aligned}
D_{\text{KL}}(q_\psi(\mathbf{z}) \parallel p(\mathbf{z} \mid \mathbf{y})) &= \mathbb{E}_{\mathbf{z} \sim q_\psi} [\log q_\psi(\mathbf{z}) - \log p(\mathbf{y} \mid \mathbf{z}) - \log p(\mathbf{z})] + \log p(\mathbf{y}) \\
&= -\mathbb{E}_{\mathbf{z} \sim q_\psi} [\log p(\mathbf{y} \mid \mathbf{z})] + D_{\text{KL}}(q_\psi(\mathbf{z}) \parallel p(\mathbf{z})) + \log p(\mathbf{y}) \\
&= -\text{ELBO}(\psi; \mathbf{y}) + \log p(\mathbf{y})
\end{aligned} \tag{2.25}$$

Since $\log p(\mathbf{y})$ does not depend on ψ , minimising the KL divergence is equivalent to maximising the ELBO, which requires only being able to evaluate the unnormalised posterior $p(\mathbf{y} \mid \mathbf{z}) p(\mathbf{z})$.

This is the same reverse-KL objective used to train normalising flows in Equation 2.15, with q_ψ in place of p_ϕ and the posterior as the target. The VI framework makes explicit that the normalising constant of the target is not required to minimise this objective. It also offers an interpretation specific to posterior inference. By Equation 2.25, maximising the ELBO balances two goals: placing mass on latents that explain the observation well, by maximising the expected log-likelihood $\mathbb{E}_{\mathbf{z} \sim q_\psi} [\log p(\mathbf{y} \mid \mathbf{z})]$, and keeping q_ψ close to the prior, by minimising $D_{\text{KL}}(q_\psi(\mathbf{z}) \parallel p(\mathbf{z}))$.

Bibliography

- [1] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. arXiv: 2006.11239 [cs.LG]. URL: <https://arxiv.org/abs/2006.11239>.
- [2] Yang Song and Stefano Ermon. *Generative Modeling by Estimating Gradients of the Data Distribution*. 2020. arXiv: 1907.05600 [cs.LG]. URL: <https://arxiv.org/abs/1907.05600>.
- [3] Tero Karras et al. *Elucidating the Design Space of Diffusion-Based Generative Models*. 2022. arXiv: 2206.00364 [cs.CV]. URL: <https://arxiv.org/abs/2206.00364>.
- [4] Yaron Lipman et al. *Flow Matching for Generative Modeling*. 2023. arXiv: 2210.02747 [cs.LG]. URL: <https://arxiv.org/abs/2210.02747>.
- [5] Xingchao Liu, Chengyue Gong, and Qiang Liu. *Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow*. 2022. arXiv: 2209.03003 [cs.LG]. URL: <https://arxiv.org/abs/2209.03003>.
- [6] Robin Rombach et al. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2022. arXiv: 2112.10752 [cs.CV]. URL: <https://arxiv.org/abs/2112.10752>.
- [7] Jonathan Ho et al. *Video Diffusion Models*. 2022. arXiv: 2204.03458 [cs.CV]. URL: <https://arxiv.org/abs/2204.03458>.
- [8] Haohe Liu et al. *AudioLDM: Text-to-Audio Generation with Latent Diffusion Models*. 2023. arXiv: 2301.12503 [cs.SD]. URL: <https://arxiv.org/abs/2301.12503>.
- [9] Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. *Stochastic Interpolants: A Unifying Framework for Flows and Diffusions*. 2025. arXiv: 2303.08797 [cs.LG]. URL: <https://arxiv.org/abs/2303.08797>.
- [10] Peter Holderrieth and Ezra Erives. *Introduction to Flow Matching and Diffusion Models*. 2026. arXiv: 2506.02070. URL: <https://diffusion.csail.mit.edu/>.

- [11] Nicholas M. Boffi, Michael S. Albergo, and Eric Vanden-Eijnden. *Flow map matching with stochastic interpolants: A mathematical framework for consistency models*. 2025. arXiv: 2406.07507 [cs.LG]. URL: <https://arxiv.org/abs/2406.07507>.
- [12] Nicholas M. Boffi, Michael S. Albergo, and Eric Vanden-Eijnden. *How to build a consistency model: Learning flow maps via self-distillation*. 2025. arXiv: 2505.18825 [cs.LG]. URL: <https://arxiv.org/abs/2505.18825>.
- [13] Yang Song et al. *Consistency Models*. 2023. arXiv: 2303.01469 [cs.LG]. URL: <https://arxiv.org/abs/2303.01469>.
- [14] Zhengyang Geng et al. *Mean Flows for One-step Generative Modeling*. 2025. arXiv: 2505.13447 [cs.LG]. URL: <https://arxiv.org/abs/2505.13447>.
- [15] Prafulla Dhariwal and Alex Nichol. *Diffusion Models Beat GANs on Image Synthesis*. 2021. arXiv: 2105.05233 [cs.LG]. URL: <https://arxiv.org/abs/2105.05233>.
- [16] Jonathan Ho and Tim Salimans. *Classifier-Free Diffusion Guidance*. 2022. arXiv: 2207.12598 [cs.LG]. URL: <https://arxiv.org/abs/2207.12598>.
- [17] Gregory Ongie et al. *Deep Learning Techniques for Inverse Problems in Imaging*. 2020. arXiv: 2005.06001 [eess.IV]. URL: <https://arxiv.org/abs/2005.06001>.
- [18] Chenlin Meng et al. *On Distillation of Guided Diffusion Models*. 2023. arXiv: 2210.03142 [cs.CV]. URL: <https://arxiv.org/abs/2210.03142>.
- [19] Huayu Chen et al. *Visual Generation Without Guidance*. 2025. arXiv: 2501.15420 [cs.CV]. URL: <https://arxiv.org/abs/2501.15420>.
- [20] Hyungjin Chung et al. *Diffusion Posterior Sampling for General Noisy Inverse Problems*. 2024. arXiv: 2209.14687 [stat.ML]. URL: <https://arxiv.org/abs/2209.14687>.
- [21] Jiaming Song et al. “Pseudoinverse-Guided Diffusion Models for Inverse Problems”. In: *International Conference on Learning Representations*. 2023. URL: https://openreview.net/forum?id=9_gsMA8MRKQ.
- [22] Bingliang Zhang et al. “Improving Diffusion Inverse Problem Solving with Decoupled Noise Annealing”. In: *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, June 2025, pp. 20895–20905. URL: <http://dx.doi.org/10.1109/CVPR52734.2025.01946>.
- [23] Jeongsol Kim, Bryan Sangwoo Kim, and Jong Chul Ye. *FlowDPS: Flow-Driven Posterior Sampling for Inverse Problems*. 2025. arXiv: 2503.08136 [cs.CV]. URL: <https://arxiv.org/abs/2503.08136>.

- [24] Abbas Mammadov et al. *Variational Flow Maps: Make Some Noise for One-Step Conditional Generation*. 2026. arXiv: 2603.07276 [cs.CV]. URL: <https://arxiv.org/abs/2603.07276>.
- [25] David M. Blei, Alp Kucukelbir, and Jon D. McAuliffe. “Variational Inference: A Review for Statisticians”. In: *Journal of the American Statistical Association* 112.518 (Apr. 2017), pp. 859–877. URL: <http://dx.doi.org/10.1080/01621459.2017.1285773>.
- [26] Danilo Jimenez Rezende and Shakir Mohamed. *Variational Inference with Normalizing Flows*. 2016. arXiv: 1505.05770 [stat.ML]. URL: <https://arxiv.org/abs/1505.05770>.
- [27] George Papamakarios et al. *Normalizing Flows for Probabilistic Modeling and Inference*. 2021. arXiv: 1912.02762 [stat.ML]. URL: <https://arxiv.org/abs/1912.02762>.
- [28] Frank Noé et al. *Boltzmann Generators – Sampling Equilibrium States of Many-Body Systems with Deep Learning*. 2019. arXiv: 1812.01729 [stat.ML]. URL: <https://arxiv.org/abs/1812.01729>.
- [29] Ian J. Goodfellow et al. *Generative Adversarial Networks*. 2014. arXiv: 1406.2661 [stat.ML]. URL: <https://arxiv.org/abs/1406.2661>.
- [30] Jerry Y. Huang et al. *How to Guide Your Flow: Few-Step Alignment via Flow Map Reward Guidance*. 2026. arXiv: 2604.27147 [cs.LG]. URL: <https://arxiv.org/abs/2604.27147>.
- [31] Peter Potapchik et al. *Meta Flow Maps enable scalable reward alignment*. 2026. arXiv: 2601.14430 [stat.ML]. URL: <https://arxiv.org/abs/2601.14430>.
- [32] Peter Holderrieth et al. *Diamond Maps: Efficient Reward Alignment via Stochastic Flow Maps*. 2026. arXiv: 2602.05993 [cs.LG]. URL: <https://arxiv.org/abs/2602.05993>.
- [33] Sander Dieleman. *Learning the integral of a diffusion model*. 2026. URL: <https://sander.ai/2026/05/06/flow-maps.html>.
- [34] Shuangfei Zhai et al. *Normalizing Flows are Capable Generative Models*. 2025. arXiv: 2412.06329 [cs.CV]. URL: <https://arxiv.org/abs/2412.06329>.
- [35] Mathieu Germain et al. *MADE: Masked Autoencoder for Distribution Estimation*. 2015. arXiv: 1502.03509 [cs.LG]. URL: <https://arxiv.org/abs/1502.03509>.
- [36] Diederik P. Kingma et al. *Improving Variational Inference with Inverse Autoregressive Flow*. 2017. arXiv: 1606.04934 [cs.LG]. URL: <https://arxiv.org/abs/1606.04934>.

- [37] George Papamakarios, Theo Pavlakou, and Iain Murray. *Masked Autoregressive Flow for Density Estimation*. 2018. arXiv: 1705.07057 [stat.ML]. URL: <https://arxiv.org/abs/1705.07057>.
- [38] Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023. URL: <http://probml.github.io/book2>.

Appendix A

Supplementary Material

A.1 Change of Variables

A.2 Score functions and score matching

A.3 Additional Experiments

A.3.1 GMM Experiments

Capacity Reward Hack in SR Inverse Problem